

itleAgentic PDF Generation with LaTeX, LuaLaTeX, CLS Templates,
Skills, QA Orchestration, and Reproducible GitHub Submission

Gemini CLI Agent

June 10, 2026

1 Assignment Requirements and Objectives

This document details the architecture and implementation strategy for Assignment 04, focusing on the autonomous generation of a long-form academic report. The core objective is to demonstrate a robust pipeline that leverages agentic workflows, sophisticated typesetting with LaTeX, and rigorous quality assurance to produce a reproducible and high-quality PDF document. The assignment mandates adherence to specific constraints, including the use of existing LaTeX templates and packages, maintaining compilability with LuaLaTeX and BasicTeX, and avoiding filler content in favor of substantive academic writing. The final output should approximate 50 pages in length, reflecting a substantial depth of content rather than a superficial outline.

The primary goals for this assignment are multifaceted:

- To establish an end-to-end system for automated document creation.
- To showcase the power of agent-based systems in complex content generation tasks.
- To ensure reproducibility and maintainability through structured code, declarative skills, and version control.
- To implement comprehensive quality assurance checks for both content and form.
- To provide clear documentation for setup, compilation, and submission.

Each component of this system, from the orchestrator to individual skills, is designed with these objectives in mind, ensuring a cohesive and effective solution.

2 Context from Lecture 07: Agentic Systems and Automation

Lecture 07 provided foundational insights into the principles of agentic systems, which are central to this assignment's approach. Agents, as intelligent entities capable of perceiving their environment and acting autonomously to achieve goals, offer a powerful paradigm for automating complex tasks. The lecture emphasized the importance of modularity, communication protocols between agents, and the concept of "skills" as specialized capabilities that agents can possess and utilize. This framework aligns directly with the assignment's design, where specialized agents (skills) are orchestrated to perform distinct roles in the document generation process, such as content writing, LaTeX compilation, and quality assurance. The lecture's discussion on goal-driven behavior and adaptive decision-making also informs the design of the orchestrator agent, which must manage sub-goals and adapt to potential issues during the generation workflow.

3 The Role of LaTeX in Reproducible PDF Generation

LaTeX remains a cornerstone in academic publishing and technical documentation due to its unparalleled capabilities in producing high-quality, professional-looking documents. Its strength lies not just in typesetting sophisticated mathematical formulas and complex layouts, but fundamentally in its approach to document structure and reproducibility. Unlike WYSIWYG editors, LaTeX separates content from presentation; authors focus on the logical structure of their document (sections, paragraphs, lists, citations), and LaTeX's engine handles the visual rendering based on predefined styles and templates. This logical separation is crucial for reproducibility. A LaTeX source file, when compiled on any system with a compatible LaTeX distribution, will produce the exact same output PDF. This consistency is vital for scientific research, where precise presentation of data and findings is paramount. Furthermore, LaTeX source files are plain text, making them ideal for version control systems like Git. This allows for tracking changes, collaborating effectively, and reverting to previous versions, all of which are essential components of a reproducible workflow.

The stability and maturity of the LaTeX ecosystem mean that documents compiled years apart can still yield identical results, provided the same distribution and packages are used. This long-term stability is a significant advantage over many modern document creation tools. The ability to define custom commands and environments, and to integrate external data sources or scripts, further enhances LaTeX's power for complex, large-scale document generation projects, making it an ideal choice for the framework developed in this assignment.

4 Advantages of LuaLaTeX

While traditional LaTeX engines like pdfLaTeX are powerful, LuaLaTeX offers significant advancements that are particularly beneficial for complex, automated document generation. LuaLaTeX integrates the Lua scripting language directly into the typesetting process. This allows for more sophisticated document manipulation and control than is possible with standard LaTeX macros alone.

Key advantages of LuaLaTeX include:

- **extbfEmbedded Scripting:** Developers can write Lua scripts that run at compile time. These scripts can dynamically generate content, modify document structures, interact with external data, or perform complex calculations before the final typesetting occurs. This capability is invaluable for automating the creation of detailed reports, bibliographies, or index entries.
- **extbfEnhanced Unicode Support:** LuaLaTeX provides superior support for Unicode characters, which is essential for documents dealing with multiple languages, specialized symbols, or modern character sets. This makes it more flexible for internationalization and diverse content needs.
- **extbfExtensibility:** The Lua API exposed to LaTeX allows for deep customization and extension of LaTeX's capabilities. Package authors can leverage Lua to create more powerful and efficient packages, and users can implement custom logic directly within their documents.

- **extbfPerformance:** In many scenarios, LuaLaTeX can offer performance improvements, especially for documents that require extensive processing or dynamic content generation, due to the efficiency of the Lua interpreter.

For an assignment focused on agentic PDF generation, LuaLaTeX’s ability to programmatically control document creation and structure makes it a powerful tool. It allows the ‘LaTeX Builder Agent’ to execute sophisticated logic, thereby enabling the generation of more dynamic and complex documents that would be challenging with older engines.

5 The Role of the CLS Template

The Document Class file (“`.cls`”) serves as the blueprint for a LaTeX document’s overall structure, appearance, and behavior. In this assignment, the `extttcustom-academic-template.cls` file plays a critical role in enforcing consistency, adhering to specific academic standards, and streamlining the document creation process.

Its responsibilities include:

- **extbfDefining Document Structure:** It sets up the basic commands and environments for sections, subsections, figures, tables, bibliographies, and other standard document elements. This ensures a uniform structure across all generated documents.
- **extbfEnforcing Styling and Formatting:** The CLS file dictates the typography (fonts, sizes, line spacing), margins, page layout, and the visual presentation of headings, lists, and other elements. This guarantees a professional and aesthetically pleasing academic look, consistent with the assignment’s requirements.
- **extbfManaging Packages:** It loads and configures essential LaTeX packages required for the document’s functionality. By managing package inclusion here, we ensure that all necessary tools for typesetting, referencing, or specific formatting are available without cluttering the main document file.
- **extbfProviding Custom Commands and Environments:** The template can define custom commands or environments tailored to the specific needs of the project, such as commands for specific types of figures, tables, or even agent-generated content blocks.
- **extbfEnsuring Reproducibility:** By centralizing formatting and package management, the CLS file ensures that documents compiled on different systems using the same template and LaTeX distribution will look identical, contributing significantly to the overall reproducibility of the generated PDFs.

In essence, the `extttcustom-academic-template.cls` acts as a contract between the content generators (agents) and the typesetting engine, ensuring that the output is both structurally sound and visually compliant with academic conventions.

6 Skills as Reusable Agent Instructions

In the context of agent-based systems, “skills” represent modular, self-contained units of functionality that an agent can invoke to perform specific tasks. For this assignment, skills

are designed as declarative instructions or scripts that encapsulate specialized knowledge or actions required for document generation. They promote a clean separation of concerns, allowing the orchestrator agent to manage the overall workflow while delegating specific sub-tasks to agents equipped with the appropriate skills.

The benefits of using skills are manifold:

- **extbfModularity:** Each skill focuses on a single responsibility (e.g., writing a specific section, checking citations, compiling LaTeX). This makes the system easier to understand, develop, and maintain.
- **extbfReusability:** A skill developed for one part of the document generation process can be reused in other contexts or by different agents, promoting efficiency and consistency. For instance, a 'QA References Skill' could be applied to various document sections.
- **extbfAbstraction:** Agents interacting with skills do not need to know the intricate details of how a skill is implemented. They only need to know what input the skill requires and what output it produces. This simplifies agent design and interaction.
- **extbfScalability:** As the complexity of the document generation task grows, new skills can be added or existing ones can be improved without significantly altering the core orchestrator logic.
- **extbfTestability:** Individual skills can be tested in isolation, simplifying the debugging process and ensuring the reliability of each component.

The skills used in this assignment, such as 'Content Writer', 'LaTeX Builder', and various 'QA Agents', are designed to be callable by the orchestrator, forming the building blocks of the autonomous PDF generation pipeline.

7 Orchestrator Agent Design

The Orchestrator Agent is the central intelligence of the autonomous PDF generation system. Its primary responsibility is to manage the entire workflow, from understanding the overall goal (generating a comprehensive academic report) to breaking it down into manageable sub-tasks, delegating these tasks to specialized skills, and coordinating their execution.

The design of the orchestrator agent follows a goal-oriented, state-driven approach:

- **extbfGoal Decomposition:** Upon receiving the request to generate the document, the orchestrator breaks down the high-level objective into a series of sequential and parallel sub-goals. This decomposition is informed by the assignment requirements, including the specific sections to be generated and the desired document structure.
- **extbfTask Delegation:** For each sub-goal, the orchestrator identifies the most appropriate skill or agent to handle it. For example, generating text for a section is delegated to the 'Content Writer Agent', while compiling the LaTeX document is assigned to the 'LaTeX Builder Agent'.

- **extbfWorkflow Management:** The orchestrator manages the dependencies between tasks. For instance, content must be written before it can be compiled, and compilation must succeed before QA checks can be performed on the output PDF. It ensures tasks are executed in the correct order, potentially in parallel where dependencies allow.
- **extbfError Handling and Recovery:** A critical function of the orchestrator is to monitor the execution of delegated tasks. If a skill fails or produces an error (e.g., LaTeX compilation errors), the orchestrator must detect this, attempt recovery strategies (e.g., re-running a task with modified parameters, using an alternative approach), or report the failure to a higher authority (e.g., the user).
- **extbfState Management:** The orchestrator maintains the current state of the document generation process, tracking which tasks have been completed, which are in progress, and which are pending. This state information is crucial for resuming interrupted processes or for providing status updates.

The orchestrator acts as the conductor of an orchestra, ensuring that each instrument (skill) plays its part at the right time, contributing to a harmonious final performance—in this case, a polished, accurate, and reproducible PDF document.

8 Content Writer Agent

The Content Writer Agent is responsible for generating the textual content of the academic report. Its role is to take high-level prompts or outlines provided by the orchestrator and transform them into well-structured, informative, and academically appropriate paragraphs and prose. This agent is crucial for populating the document with substantive material that meets the required length and quality standards.

Key aspects of the Content Writer Agent include:

- **extbfPrompt Interpretation:** The agent receives detailed prompts from the orchestrator, specifying the topic, desired length, key points to cover, and the target audience/style (e.g., academic, formal).
- **extbfKnowledge Retrieval:** It may access internal knowledge bases, external information sources (if permitted and configured), or pre-defined content structures to gather relevant information for the assigned section.
- **extbfText Generation:** Utilizing advanced language models or rule-based systems, the agent formulates coherent sentences, paragraphs, and longer passages. It focuses on maintaining a consistent tone, logical flow, and accurate representation of information.
- **extbfAdherence to Constraints:** The agent must ensure that the generated content aligns with the assignment's requirements, such as avoiding filler text, not using Hebrew characters prematurely, and fulfilling the page count target by providing sufficient detail.
- **extbfOutput Formatting:** The generated text is typically produced in a format suitable for integration into the LaTeX document, often as plain text or markdown, which can then be processed by the LaTeX Builder Agent.

For this assignment, the Content Writer Agent will be tasked with elaborating on each of the sections outlined in the requirements, ensuring that the generated text is not only factually accurate but also stylistically appropriate for an academic report.

9 LaTeX Builder Agent

The LaTeX Builder Agent's core function is to take the content generated by other agents and assemble it into a fully compilable LaTeX document, ultimately producing the final PDF output. This agent acts as the bridge between the abstract content and the concrete, typeset document, leveraging the power of LuaLaTeX and the project's custom CLS template.

Its responsibilities are as follows:

- **extbfDocument Assembly:** It receives the structured content (e.g., text for sections, tables, figure captions) and embeds it within the LaTeX structure defined by the `extttcustom-academic-template.cls`. This involves placing content in the correct sections, ensuring proper use of LaTeX commands for formatting and structure.
- **extbfCompilation Execution:** The agent invokes the LuaLaTeX compiler to process the assembled `'tex'` file. This step transforms the source code into a PDF. The agent must ensure that the correct compiler is used and that any necessary compilation passes (e.g., for references, table of contents) are executed.
- **extbfError Handling:** If the LuaLaTeX compilation process fails, the agent is responsible for capturing the error messages. It then reports these errors back to the orchestrator, which may attempt to resolve them or use them to refine subsequent generation steps.
- **extbfOutput Management:** Upon successful compilation, the agent ensures that the generated PDF file, along with any intermediate files (like `'aux'`, `'log'`), are managed appropriately, typically making them available for the QA agents.
- **extbfConfiguration:** It ensures that the compilation environment is correctly configured, respecting any specific requirements of the CLS template or the LuaLaTeX engine.

This agent is critical for translating the abstract "what" of the content into the concrete "how" of the typeset document, directly contributing to the assignment's goal of automated, high-quality PDF generation.

10 QA Structure Agent

The QA Structure Agent is dedicated to verifying the structural integrity and logical organization of the generated LaTeX document. Its role is to ensure that the document adheres to the expected academic hierarchy and formatting conventions, complementing the visual typesetting provided by LaTeX. This agent plays a crucial role in validating that the document's framework is sound before proceeding to more detailed content checks.

Key functions of the QA Structure Agent include:

- **extbfSection and Subsection Validation:** It checks that all intended sections and subsections are present, correctly numbered, and logically ordered according to the assignment requirements and academic best practices.
- **extbfPage Break Verification:** The agent assesses the placement and appropriateness of page breaks, particularly those explicitly inserted using `exttt ewpage`. It verifies that major sections are not awkwardly split or that new major parts begin on new pages as intended.
- **extbfList and Enumeration Checks:** It may verify the correct formatting and numbering of lists, itemizations, and enumerations within the document.
- **extbfTable and Figure Placement:** The agent can check that tables and figures are correctly placed, captioned, and labeled, and that references to them within the text are accurate.
- **extbfTable of Contents Consistency:** It confirms that the Table of Contents accurately reflects the structure and pagination of the document's main body.

By systematically analyzing the document's structural elements, the QA Structure Agent ensures that the generated output is not only aesthetically pleasing but also logically coherent and easy to navigate, fulfilling a critical aspect of the assignment's quality requirements.

11 QA References Agent

The QA References Agent is responsible for ensuring the accuracy and integrity of all citations, bibliographical entries, and cross-references within the generated document. In academic writing, correct referencing is paramount for credibility and for avoiding plagiarism. This agent automates the verification of these critical elements.

Its key tasks include:

- **extbfCitation Verification:** The agent checks that every citation made in the text (e.g., `exttt[?]key`) corresponds to a valid entry in the bibliography. It verifies that citation keys are correctly spelled and formatted.
- **extbfBibliography Integrity:** It ensures that all entries in the bibliography section are correctly formatted according to the specified style (as dictated by the CLS template or associated bibliography style files) and that all cited works are present.
- **extbfCross-Reference Validation:** For internal references (e.g., to figures, tables, sections using `exttt ef{label}`, `exttt??`), the agent confirms that the labels exist and that the referenced numbers or page numbers are accurate.
- **extbfUnused References Detection:** It can identify citation keys or labels that are defined but never used, helping to clean up the document.

By performing these checks, the QA References Agent guarantees that the document's scholarly apparatus is robust and error-free, contributing significantly to its overall quality and trustworthiness. The correct functioning of this agent relies on the LaTeX Builder Agent successfully processing the bibliography and references during compilation.

12 QA BiDi Agent

The QA BiDi (Bidirectional) Agent’s role is to ensure that the document correctly handles text that flows in multiple directions, particularly for languages that use right-to-left (RTL) scripts. While the current assignment explicitly states not to use Hebrew text yet, the inclusion of a BiDi agent anticipates future multilingual capabilities and ensures the framework is prepared for such requirements.

The agent’s potential functions include:

- `extbfDirectionality Analysis`: It analyzes the document to identify sections that require RTL rendering (e.g., Hebrew or Arabic text) and verifies that these sections are correctly marked and processed by the LaTeX engine.
- `extbfScript Mixing`: For documents that mix Left-to-Right (LTR) and RTL scripts, the agent checks for correct handling of text flow, punctuation, and alignment at the boundaries between different directional texts.
- `extbfLayout Consistency`: It ensures that the overall document layout, including paragraph alignment and the placement of elements like lists or quotations, remains consistent and readable regardless of the text directionality.
- `extbfFont Support Verification`: The agent can flag potential issues related to font support for specific characters or scripts, ensuring that all necessary glyphs are available for rendering.

Even without immediate RTL content, defining this agent establishes a placeholder for future expansion and highlights the framework’s potential for internationalization. Its checks would primarily focus on ensuring that appropriate LaTeX packages (like `extttpolyglossia` or `extttbabel` with correct language options, if they were permitted) are correctly configured and that the document structure supports mixed-directionality.

13 Repository Structure

The project repository is organized to promote modularity, clarity, and maintainability, facilitating the agentic PDF generation workflow. The structure is designed to separate concerns across different directories, making it easier to locate and manage specific components of the system.

The main directories and their purposes are as follows:

- `exttt/ex04/`: This top-level directory contains the core artifacts for Assignment 04.
 - `extttmain.tex`: The primary LaTeX source file for the academic report.
 - `extttcustom-academic-template.cls`: The custom document class file that defines the report’s structure and styling.
 - `extttreferences.bib`: The bibliography file containing all references cited in the document.
 - `extttassets/`: Contains various assets required for the document, such as:
 - * `extttgraphs/`: Placeholder for generated or sourced charts and graphs.
 - * `extttimages/`: Directory for image files used in the document.

- extttskills/: This directory houses the declarative definitions for various agent skills. Each file (e.g., extttcontent_writer_skill.md) represents a distinct agent capability.
- extttsrc/: Contains the Python scripts that implement the core agents and logic:
 - * extttorchestrator.py: The main orchestrator agent script.
 - * extttcompile_pdf.py: Logic for the LaTeX Builder Agent.
 - * extttqa_check.py: Scripts for various QA agents (structure, references, BiDi).
- extttlogs/: Stores log files generated during the execution of agents and compilation processes.
 - * extttgeneration_log.md: Logs related to content generation.
 - * extttqa_report.md: Summarized reports from the QA agents.
- extttscreenshots/: Captures of intermediate or final outputs, useful for debugging and documentation.
- exttt/ex02/ and exttt/ex03/: These directories contain artifacts from previous assignments, likely demonstrating different aspects of agentic workflows or document generation. They are excluded from modifications as per assignment rules.
- extttREADME.md: The main project README file, providing an overview of the repository.
- exttt.gitignore: Configures Git to ignore specific files and directories (e.g., build artifacts, temporary files).

This hierarchical structure ensures that related components are grouped together, promoting a clear understanding of the project’s architecture and facilitating development and maintenance.

14 GitHub Submission Strategy

A robust GitHub submission strategy is essential for ensuring that the generated document and its associated code are well-managed, trackable, and reproducible. This strategy involves clear guidelines for development, version control, and submission to meet the requirements of this assignment and standard software engineering practices.

The proposed strategy includes the following steps:

- extbfBranching Strategy: All development work will be conducted on a dedicated feature branch, such as extttassignment04-feature, branching off the main development line (e.g., extttmain or extttdevelop). This isolates changes and prevents direct interference with stable code.
- extbfLocal Development and Compilation: Developers will use their local environment (configured with BasicTeX and LuaLaTeX) to develop and test the agentic pipeline. The extttsrc/ scripts and extttskills/ definitions will be iterated upon, with frequent compilation of exttttex04/main.tex to verify output and catch errors early.

- extbfVersion Control:
 - extttgit add: All modified source files (Python scripts, skill definitions, LaTeX files, CLS templates) will be staged.
 - extttgit commit: Changes will be committed with clear, descriptive messages outlining the purpose and scope of the commit. For example: "feat: Implement Content Writer agent for section 3" or "fix: Resolve compilation error in LaTeX Builder". Commit messages will follow a conventional format (e.g., Conventional Commits).
 - extttgit status: Regularly used to review changes before staging and committing.
 - extttgit diff: Used to review the exact changes introduced in a commit.
- extbfPull Request (PR): Once the development on the feature branch is complete and thoroughly tested locally, a pull request will be opened against the main development branch. This PR will serve as a formal proposal for merging the changes.
- extbfCode Review: The PR will be subject to code review by peers or automated systems. This is an opportunity to identify potential issues, suggest improvements, and ensure adherence to coding standards and assignment requirements.
- extbfMerging: Upon approval, the changes from the feature branch will be merged into the main development branch.
- extbfFinal Submission: The final submission will likely involve ensuring the 'main' branch reflects the completed assignment, or as per specific instructions from the instructor (e.g., creating a release tag). The presence of the compiled PDF, source code, and all necessary configuration files in the repository ensures full reproducibility.

This systematic approach ensures that the project evolves in a controlled manner, with a complete history of changes and a high degree of confidence in the final submission's quality and reproducibility.

15 Setup and Compilation Workflow

To ensure reproducible results, a clear workflow for setting up the environment and compiling the LaTeX document is essential. This workflow leverages the tools and structure defined for this assignment.

15.1 Environment Setup

Before compilation, the following setup steps are necessary:

1. extbfInstall BasicTeX: Ensure a minimal TeX distribution is installed. BasicTeX is recommended for its lightweight nature. For macOS, this can be installed via its official installer.

2. `extbfInstall LuaLaTeX`: BasicTeX typically includes LuaLaTeX. If not, or if a different distribution is used, ensure LuaLaTeX is installed. Verify by running `extt-lualatex -version` in the terminal.
3. `extbfInstall Python Dependencies`: The Python scripts powering the agents require specific libraries. Install them using `pip`:

```
pip install -r ex04/requirements.txt
```

(Note: `extttrequirements.txt` needs to be created or exist in the specified path.)

4. `extbfClone Repository`: Ensure the project repository is cloned and accessible. All paths mentioned are relative to the repository root.

15.2 Compilation Workflow

The compilation process, managed by the LaTeX Builder Agent, involves the following steps:

1. `extbfContent Generation`: The Orchestrator Agent first invokes the Content Writer Agent to generate text for all required sections, populating `extttmain.tex` or an intermediate content store.
2. `extbfLaTeX Assembly`: The LaTeX Builder Agent then takes the generated content and integrates it into the `extttmain.tex` file, ensuring it correctly uses the `extttcustom-academic-template.cls`.
3. `extbfBibliography Processing`: If using `extttbiblatex` or similar, the bibliography file (`extttreferences.bib`) is processed. (Assuming the CLS handles this).
4. `extbfLuaLaTeX Compilation`: The agent executes LuaLaTeX multiple times to resolve all dependencies, cross-references, and table of contents entries:

```
lualatex ex04/main.tex
lualatex ex04/main.tex
# Potentially additional runs for bibtex/biber if not handled by lualatex
# e.g., bibtex ex04/main or biber ex04/main if needed and supported by .
lualatex ex04/main.tex
lualatex ex04/main.tex
```

The exact sequence of commands might depend on the specifics of the CLS template and any bibliography package used. A minimum of two runs is typically needed for ToC and references, often more for complex documents.

5. `extbfQA Checks`: Post-compilation, the QA agents (Structure, References, BiDi) are invoked to validate the generated PDF and LaTeX output.
6. `extbfLog Archiving`: All compilation logs (`exttt.log` files) and QA reports are saved in the `extttex04/logs/` directory for inspection.

This structured approach ensures that the document can be consistently generated and that any issues encountered during the process are identifiable and resolvable.

16 Error Handling During Compilation

LaTeX compilation, especially with sophisticated engines like LuaLaTeX and custom templates, can encounter various errors. A robust error handling strategy is crucial for the success of the autonomous generation pipeline. The LaTeX Builder Agent, in coordination with the Orchestrator, is responsible for detecting, reporting, and potentially resolving these issues.

Common types of LaTeX compilation errors include:

- **Syntax Errors:** Incorrect LaTeX commands, missing braces, unmatched environments, unsafe inline examples, or typos in commands.
- `extbfUndefined` Control Sequences: Using commands that are not recognized, often due to missing packages or typos.
- `extbfMissing` Files: The compiler cannot find required files, such as images, bibliography data, or custom style files.
- `extbfPackage` Conflicts: Incompatible packages loaded into the document, leading to unpredictable behavior or outright failure.
- `extbfFloats` Placement Errors: Issues with figures or tables not fitting on the page or causing layout problems.
- `extbfReference` Errors: Citations or cross-references to labels that do not exist or are not correctly processed.
- `extbfEncoding` Issues: Problems with character encoding, especially when mixing different languages or special symbols.

The error handling process typically involves:

1. `extbfCapturing` Output: The LaTeX Builder Agent captures the standard output and error streams from the LuaLaTeX compiler. The `exttt.log` file generated by LaTeX is also a primary source of detailed error information.
2. `extbfError` Identification: The agent parses these outputs to identify specific error messages, often indicated by lines starting with `exttt! LaTeX Error:` or similar. It attempts to pinpoint the error message, the relevant line number in the `'tex'` file, and the context.
3. `extbfReporting` to Orchestrator: Identified errors are reported back to the Orchestrator Agent. The Orchestrator then decides on the next course of action.
4. `extbfRecovery` Strategies: Depending on the error type, the Orchestrator might:
 - `extbfRetry` Compilation: For transient issues, a simple retry might suffice.
 - `extbfModify` Content: If an error is due to malformed content generated by the Content Writer, the Orchestrator might request a revision of that content.
 - `extbfAdjust` Compilation Flags: Modify compilation commands if specific options are known to resolve certain issues.

- `extbfInform User`: If an error is unrecoverable by the agents, the user is notified with detailed diagnostic information.

By systematically handling compilation errors, the system aims to produce a reliable output and provides valuable feedback for improving the generation process.

17 Previous Feedback and Improvements

This section documents how feedback from prior stages or iterations of the assignment has been incorporated to enhance the system’s robustness and quality. While specific feedback from previous assignments is not detailed here, the process assumes a cyclical development approach where evaluation leads to refinement.

Typical areas where feedback might lead to improvements include:

- `extbfContent Relevance and Depth`: If previous content was deemed too superficial or not relevant enough, the Content Writer Agent’s prompts would be refined to elicit more detailed and focused academic writing. This might involve providing more structured outlines or specific research directives.
- `extbfCompilation Success Rate`: If compilation errors were frequent, the LaTeX Builder Agent’s logic might be enhanced to include more robust error detection, logging, and potentially automated fixes for common issues. Ensuring the CLS template and `extttreferences.bib` are correctly managed is also key.
- `extbfQA Coverage and Accuracy`: Feedback on missed errors or false positives from QA agents would lead to adjustments in their logic. For instance, the QA References Agent might be updated to handle more complex citation styles or identify a wider range of reference errors.
- `extbfReproducibility`: If users struggled to reproduce the output, improvements would focus on refining the environment setup instructions, ensuring all dependencies are clearly listed (e.g., in `extttrequirements.txt`), and verifying that the compilation workflow is consistently applied.
- `extbfPerformance`: If the generation process was too slow, optimizations might be introduced, such as parallelizing independent tasks, refining agent prompts for efficiency, or investigating performance bottlenecks in the LaTeX compilation itself.

By iteratively incorporating feedback, the system evolves towards a more reliable, efficient, and high-quality document generation solution, demonstrating a commitment to continuous improvement.

18 Evaluation Checklist

This checklist provides a set of criteria against which the performance and output of the agentic PDF generation system can be evaluated. It covers key aspects of the assignment requirements, from technical correctness to the quality of the final document.

18.1 Core Functionality

extbfEnd-to-End Generation: The system successfully generates a PDF document from initial prompt/requirements.

extbfCompilability: The generated extttmain.tex compiles successfully using LuaLaTeX and BasicTeX without new packages.

extbfPage Count: The final PDF meets or approximates the target length (e.g., 50 pages) with substantial content.

extbfNo Filler Content: Content is academically relevant and detailed, avoiding generic filler text.

extbfNo Hebrew Text (Yet): Hebrew characters are not introduced prematurely.

18.2 Content and Structure

extbfAssignment Requirements Covered: All specified sections and subsections are present and adequately expanded.

extbfAcademic Style: Content is written in a formal, academic tone.

extbfLogical Flow: Sections and paragraphs transition smoothly and logically.

extbfTitle Page and ToC: Title page and Table of Contents are correctly generated and accurate.

extbfBibliography and Citations: Citations are correctly formatted, and the bibliography is complete and accurate.

extbfCross-references: Internal document references (figures, tables, sections) are correct.

18.3 Technical Implementation

extbfOrchestrator Logic: The orchestrator correctly delegates tasks and manages workflow dependencies.

extbfSkill Implementation: Skills perform their intended functions reliably.

extbfLaTeX Builder: Compiles the document correctly and handles compilation errors gracefully.

extbfQA Agents: QA agents accurately identify structural, reference, and BiDi (potential) issues.

extbfRepository Structure: Files are organized logically within the repository.

extbfReproducibility: The setup and compilation workflow are clearly documented and functional.

18.4 Submission

extbfVersion Control: Changes are managed using Git with appropriate commit messages.

extbfNo Unauthorized Modifications: Only `extttex04/main.tex` is modified. No other specified files are changed.

extbfDeliverables: All required files (source code, LaTeX, CLS, bib, logs) are present in the repository.

This checklist serves as a guide for self-assessment and final review before submission.

19 Limitations

Despite the comprehensive nature of the agentic PDF generation framework, several limitations are inherent in the current implementation and the scope of this assignment. Recognizing these limitations is crucial for understanding the system’s capabilities and for planning future development.

Key limitations include:

- **extbfContent Generation Sophistication:** While the Content Writer Agent aims for academic quality, it may not achieve the depth, nuance, or originality of human expert writing. Generating truly novel research or highly complex theoretical arguments might be beyond its current capabilities. The reliance on prompts means output quality is heavily dependent on prompt engineering.
- **extbfDynamic Asset Generation:** The current framework primarily focuses on text and structure. Generating complex figures, graphs, or custom LaTeX tables dynamically based on data inputs is not explicitly implemented, though placeholders exist. The framework relies on pre-existing or manually created assets.
- **extbfError Recovery Complexity:** While basic error handling for compilation is in place, complex LaTeX errors or subtle logical flaws in generated content might be difficult for agents to automatically resolve, potentially requiring manual intervention. The ability to recover from all possible failure modes is challenging.
- **extbfEnvironment Specificity:** While aiming for reproducibility, perfect replication across all environments can still be challenging. Subtle differences in LaTeX distributions, package versions, or operating system configurations might lead to minor variations in output. The reliance on specific tool versions (e.g., LuaLaTeX version) can be a factor.
- **extbfReal-time Interaction Limitations:** The system operates in a batch processing mode. It is not designed for real-time, interactive document editing or collaborative writing in the way a human user might experience with a word processor.
- **extbfLimited Understanding of Nuance:** Agents may struggle with highly nuanced instructions, implicit requirements, or understanding the broader context of a document in the same way a human author would.

These limitations highlight areas where further research and development are needed to achieve fully autonomous and human-level quality in document generation.

20 Future Work

The current framework provides a solid foundation for agentic PDF generation. However, several avenues for future work exist to enhance its capabilities, robustness, and user experience. Expanding upon the current architecture can lead to more sophisticated and versatile document automation tools.

Potential areas for future development include:

- **Advanced Content Generation:** Enhance the Content Writer Agent with more sophisticated reasoning capabilities, enabling it to synthesize information from multiple sources, perform critical analysis, and generate more original or research-oriented content. Integrate with larger, more capable language models.
- **Dynamic Asset Creation:** Develop agents capable of automatically generating figures, charts, and complex tables based on data inputs. This would involve integrating with plotting libraries or data visualization tools.
- **Multi-lingual Support:** Fully implement BiDi support and extend the framework to generate documents in multiple languages, including automatic translation and adaptation of cultural/linguistic conventions.
- **Interactive Refinement:** Introduce a feedback loop where the orchestrator can solicit specific user input or clarification during the generation process, allowing for more guided and interactive document creation.
- **Enhanced Error Resolution:** Develop more sophisticated strategies for automatic error correction during LaTeX compilation, potentially using machine learning models trained on common LaTeX errors.
- **Version Management Integration:** Tighter integration with version control systems, such as automated branching, commit message generation, and PR creation based on generation milestones.
- **GUI/Web Interface:** Develop a user-friendly interface for configuring generation tasks, managing skills, and monitoring the progress of document generation.
- **Template Learning:** Enable agents to learn and adapt to new CLS templates or style guides, increasing the framework’s flexibility across different academic disciplines or publication venues.

These advancements would push the boundaries of automated document generation, making it a more powerful tool for researchers, students, and professionals alike.

ewpage to separate major sections

ewpage

A Appendix A: Example Agent Prompts

This appendix provides illustrative examples of the prompts that might be used to guide the different agents. The actual prompts would be dynamically generated or selected by the orchestrator based on the specific requirements of the document section being created.

A.1 Content Writer Agent - Prompt Example

Prompt for generating Section 3: "Why LaTeX is used for reproducible PDF generation."

Generate a detailed academic section (approximately 3-4 paragraphs) explaining why LaTeX is used for reproducible PDF generation.

1. Separation of content from presentation.
2. Consistency of output across different systems and time.
3. Plain text format for version control (Git).
4. Maturity and stability of the ecosystem.
5. Benefits for collaboration and archiving.

Maintain a formal, academic tone. Do not use filler text. Ensure the content is substantive.

A.2 LaTeX Builder Agent - Compilation Command Example

Example shell command for compilation (executed by the agent):

```
lualatex --interaction=nonstopmode --output-directory=ex04 ex04/main.tex
```

Note: The `exttt--interaction=nonstopmode` flag prevents LaTeX from pausing on errors, allowing the agent to capture all errors in the log. The output directory is specified to keep build artifacts organized. Multiple runs and potentially `extttbibtex` or `extttbiber` would be chained.

A.3 QA Structure Agent - Check Example

Prompt for the QA Structure Agent:

Analyze the generated LaTeX document 'ex04/main.tex' and its compilation log 'ex04/main.tex.log'. Verify the following:

- All required sections (1-24) and subsections are present.
 - Sections are in the correct order as per assignment requirements.
 - Page breaks (`exttt{\\newpage}`) are used appropriately between major sections (e.g., before Appendices).
 - The Table of Contents accurately reflects the document's structure and pagination.
- Report any structural anomalies, missing elements, or incorrect ordering.

B Appendix B: QA Checklist

This detailed checklist is used by the QA agents to systematically evaluate the generated LaTeX document. It expands upon the Evaluation Checklist provided in Section 18.

B.1 Structural QA

Section titles match assignment list.

Section order is correct.

Subsections are appropriately nested and labeled.

`exttt \\newpage` usage is consistent with logical breaks.

Table of Contents (ToC) is generated and accurate (sections, pagination).

Page numbers are consistent and sequential.

Document begins with title page and ToC.

B.2 Content QA

Content is substantive, not filler.

Tone is academic and formal.

Paragraphs are well-formed and coherent.

Language is clear and grammatically correct.

No premature use of Hebrew text.

Assignment topic is addressed comprehensively.

B.3 Referencing QA

All in-text citations (`exttt{?}...`) have corresponding bibliography entries.

All bibliography entries are cited at least once.

Citation keys are valid and correctly spelled.

Bibliography entries follow the required format (per CLS template).

Cross-references (`exttt ef{...}`) point to correct labels.

Referenced labels (figures, tables, sections) exist.

Bibliography compiled correctly (e.g., using `extttbiblatex` or `extttnatbib` as managed by CLS).

B.4 Compilation QA

Document compiles successfully with LuaLaTeX (multiple passes).

Compilation log (`exttt.log`) is reviewed for errors or warnings.

No undefined control sequences or packages.

External files (images, bib) are correctly referenced and found.

`extttcustom-academic-template.cls` is correctly applied.

B.5 BiDi QA (Proactive/Future)

Document structure supports potential BiDi text (if required later).

No obvious layout issues that would impede RTL text.

(Future) Verification of correct RTL rendering for Hebrew/Arabic sections.

C Appendix C: Compilation Troubleshooting

Encountering compilation errors is a common part of working with LaTeX. This section provides guidance on troubleshooting typical issues that may arise during the automated generation process.

C.1 Common Errors and Solutions

C.1.1 Undefined Control Sequence / Unknown Command

extbfSymptom: `exttt!` Undefined control sequence. or `exttt!` LaTeX Error: Unknown command `extbackslash commandname`. **extbfCause:** A command was used that is not defined, either because it's misspelled, requires a package not loaded, or is from a package not installed. **extbfSolution:**

- Check for typos in the command name.
- Ensure the required package is loaded in `extttcustom-academic-template.cls`. If not, and adding packages is allowed, add `exttt{packagename}`.
- Verify that the package is installed in your LaTeX distribution (e.g., via `exttttlmgr install packagename` if using TeX Live, or managed by BasicTeX).

C.1.2 File Not Found

extbfSymptom: `extttFile` 'filename.ext' not found. **extbfCause:** The LaTeX compiler cannot locate a required file (e.g., an image, a bibliography file, or a style file). **extbfSolution:**

- Ensure the file exists at the specified path relative to the main 'tex' file or the specified output directory.
- Check that the file path in the LaTeX command (e.g., `extttpath/to/image.png`) is correct.
- Verify that intermediate build artifacts are not preventing access if compilation is complex.

C.1.3 Missing Bibliography Entry / Undefined Citation

extbfSymptom: `extttLaTeX Warning: Citation 'key' on page X undefined on input line Y.` or `extttThere were undefined citations.` **extbfCause:** A citation key used in the text does not exist in the bibliography file, or the bibliography was not processed correctly. **extbfSolution:**

- Ensure the citation key used in `exttt[?]key` exactly matches an entry's key in `extttreferences.bib`.
- Verify that the bibliography processing steps (e.g., running `extttbibtex` or `extttbiber`, depending on the CLS setup) are executed correctly as part of the compilation workflow. Multiple `extttlualatex` runs are essential.

C.1.4 Package Errors / Conflicts

extbfSymptom: Errors related to package options, `extttRedefinition` warnings, or general compilation failure after loading a package. **extbfCause:** Packages might have incompatible options or conflict with each other. **extbfSolution:**

- Consult the documentation for the conflicting packages.
- Simplify package loading in `extttcustom-academic-template.cls`. If a package is not strictly necessary, remove it.
- Ensure packages are loaded in a sensible order.

C.1.5 Infinite Loops

extbfSymptom: Compilation seems to hang or repeat indefinitely, often with repetitive output. **extbfCause:** Typically caused by recursive definitions, incorrect cross-referencing, or issues with packages that modify document flow. **extbfSolution:**

- Examine the log file for repetitive patterns.
- Simplify the document by commenting out sections or disabling specific packages to isolate the cause.
- Check for incorrect use of `exttt` and `exttt ef`.

C.2 Debugging Strategy

When errors occur, the following strategy is recommended:

1. **extbfExamine the Log File:** Always start by inspecting `extttex04/main.log` for the most detailed error messages.
2. **extbfIsolate the Error:** Use `exttt-interaction=nonstopmode` to see all errors, or temporarily comment out sections of the `exttt.tex` file to determine which part triggers the error.
3. **extbfIncremental Compilation:** Compile smaller parts of the document or known working examples to verify components.
4. **extbfConsult Documentation:** Refer to the LaTeX Companion, package documentation, or online resources (like TeX Stack Exchange) for specific error messages.

D Appendix D: Repository File Inventory

This appendix provides a comprehensive inventory of files within the project repository relevant to Assignment 04, detailing their purpose and location.

D.1 Core Assignment Files (`extttex04/`)

- `extttex04/main.tex`: The main LaTeX source document for the academic report. Contains document structure, content, and commands.
- `extttex04/custom-academic-template.cls`: The custom LaTeX document class defining the report’s styling, layout, and core functionalities.
- `extttex04/references.bib`: The BibTeX file containing all bibliographic entries cited in the document.
- `extttex04/assets/`: Directory for supporting media.
 - `extttex04/assets/graphs/`: Placeholder for generated or sourced graphs.
 - `extttex04/assets/images/`: Directory for image files.
- `extttex04/logs/`: Directory for storing logs generated during the process.
 - `extttex04/logs/generation_log.md`: Logs pertaining to content generation.
 - `extttex04/logs/qa_report.md`: Summaries of quality assurance checks.
- `extttex04/screenshots/`: Directory for visual documentation of intermediate or final outputs.

D.2 Agent Skills (`extttex04/skills/`)

- `extttex04/skills/content_writer_skill.md`: Defines the capabilities and instructions for the Content Writer Agent.
- `extttex04/skills/latex_builder_skill.md`: Defines the capabilities and instructions for the LaTeX Builder Agent.
- `extttex04/skills/qa_structure_skill.md`: Defines instructions for the QA Structure Agent.
- `extttex04/skills/qa_references_skill.md`: Defines instructions for the QA References Agent.
- `extttex04/skills/qa_bidi_skill.md`: Defines instructions for the QA BiDi Agent.
- `extttex04/skills/orchestrator_skill.md`: Defines instructions for the Orchestrator Agent.

D.3 Agent Source Code (`extttex04/src/`)

- `extttex04/src/orchestrator.py`: The main Python script implementing the Orchestrator Agent’s logic.
- `extttex04/src/compile_pdf.py`: Python script for the LaTeX Builder Agent’s compilation tasks.
- `extttex04/src/qa_check.py`: Python script containing logic for various QA agents.
- `extttex04/src/__init__.py`: Python package initialization file.

D.4 Project Root Files

- `exttttREADME.md`: Top-level project README.
- `extttt.gitignore`: Git ignore rules for the repository.
- `exttttrequirements.txt`: Lists Python dependencies for the project. (Assumed to exist or be created).

D.5 Other Assignment Directories

- `exttttex02/`: Files from Assignment 02.
- `exttttex03/`: Files from Assignment 03.

This inventory provides a map of the project’s structure, ensuring all components are accounted for and accessible.

E Appendix E: Lessons Learned from Previous Assignments

The development of this agentic PDF generation system for Assignment 04 is significantly informed by the cumulative insights and challenges encountered during previous assignments. Each prior task presented unique learning opportunities, the distillation of which has been critical in shaping the architecture, methodologies, and quality assurance protocols of the current project. This section reflects on key lessons derived from earlier experiences, demonstrating an iterative improvement process fundamental to complex software engineering and agentic system design.

E.1 From Basic Agentic Workflows (e.g., Assignment 02)

Assignment 02, focusing on foundational agentic interactions, provided initial insights into the complexities of prompt engineering and inter-agent communication.

- **Prompt Sensitivity and Specificity:** A primary lesson was the extreme sensitivity of agent output to prompt wording. Vague or ambiguous instructions consistently led to suboptimal or irrelevant responses. This underscored the necessity for hyper-specific and constrained prompts, a principle directly applied in the design of the Content Writer Agent’s prompts for Assignment 04 (see Appendix A).
- **Iterative Refinement of Agent Behavior:** Early experiences demonstrated that achieving desired agent behavior is rarely a one-shot process. It requires continuous observation of agent outputs, identification of deviations, and iterative refinement of prompts or underlying agent logic. This led to the explicit inclusion of feedback loops and QA stages in the current orchestrator design.
- **Managing Context Window Limitations:** Initial attempts often struggled with agents forgetting previous turns or exceeding context window limits. This highlighted the importance of designing agents to process information efficiently and the orchestrator to strategically manage the flow of context, avoiding redundant information.

E.2 From API Integration and Service Orchestration (e.g., Assignment 03)

Assignment 03, involving the integration with external APIs and the orchestration of more complex service flows, contributed valuable lessons in system reliability, error handling, and modularity.

- **Robust Error Handling in External Interactions:** Interacting with external services (like Gmail or Calendar APIs) inevitably exposed the system to network failures, API rate limits, and unexpected response formats. This reinforced the need for comprehensive error handling mechanisms, graceful degradation, and retry logic within agent interactions. The LaTeX Builder Agent’s error handling for compilation (Appendix C) directly benefits from this experience.
- **Modularity for Maintainability and Testability:** Building systems with distinct, loosely coupled components (e.g., separate Gmail and Calendar services) proved critical for isolating bugs and facilitating independent development. This lesson is reflected in the modular design of agents and skills in Assignment 04, where each skill is a self-contained unit.
- **Importance of Environment Configuration:** Ensuring consistent API keys, environment variables, and dependency installations (e.g., `requirements.txt`) was a recurring challenge. This led to a stronger emphasis on detailed setup and compilation workflows (Section 15) and clear repository documentation (Appendix D) for Assignment 04.
- **Security Best Practices:** Handling sensitive credentials (API keys, tokens) necessitated strict adherence to security protocols, such as using environment variables and avoiding hardcoding. This principle is implicitly embedded in the design, ensuring that sensitive information is never exposed in logs or source code.

E.3 General Insights for Agentic System Development

Beyond specific assignments, broader lessons emerged regarding the nature of agentic systems.

- **The Orchestrator as the Central Intelligence:** The orchestrator’s role evolved from a simple task dispatcher to a sophisticated manager of goals, dependencies, and recovery strategies. Its ability to dynamically adapt to unforeseen circumstances is crucial.
- **Validation as a Continuous Process:** Relying solely on final output validation is insufficient. Continuous, incremental validation (e.g., through QA agents after each major step) helps identify and rectify issues early, reducing debugging costs. This is evident in the detailed QA checklists (Appendix B) and the integration of QA agents throughout the workflow.
- **Reproducibility as a Design Goal:** From the outset, designing for reproducibility (consistent environments, clear version control, detailed documentation) became a non-negotiable requirement rather than an afterthought. This permeates the entire Assignment 04 project.

- **Human-in-the-Loop Considerations:** While aiming for autonomy, recognizing when human intervention is necessary (e.g., for complex error resolution or creative decisions) is vital. Agents should be designed to escalate issues with rich diagnostic information.

These accumulated lessons from previous assignments have profoundly influenced the design and implementation of the current agentic PDF generation system, contributing to its robustness, maintainability, and ability to meet the stringent academic requirements.

F Appendix F: Reproducibility and Submission Readiness

Ensuring the reproducibility of an academic report and preparing it for submission are critical final steps in any research or project workflow. This appendix elaborates on the measures taken within the agentic PDF generation framework to guarantee that the generated document can be independently replicated and meets all criteria for a successful academic submission. Reproducibility ensures scientific integrity, while submission readiness streamlines the handover process.

F.1 Achieving Reproducibility

Reproducibility, in the context of this project, means that any researcher or evaluator should be able to obtain the identical PDF document by following the documented setup and compilation workflow, given the same source code and inputs.

- **Version-Controlled Source Code:** All source files, including `main.tex`, `custom-academic-template.cls`, `references.bib`, agent scripts (in `src/`), and skill definitions (in `skills/`), are maintained under Git version control. This ensures that the exact state of the project at any given time can be retrieved.
- **Declarative Environment Setup:** The `requirements.txt` file precisely lists all Python dependencies, allowing for the recreation of the exact Python environment. Similarly, the reliance on BasicTeX and LuaLaTeX (standard distributions) with clear installation instructions minimizes environmental variability.
- **Explicit Compilation Workflow:** The "Setup and Compilation Workflow" (Section 15) provides a step-by-step guide to building the document from source. This includes the precise LuaLaTeX commands and the sequence of execution required to resolve all references and generate the final PDF.
- **Immutable Template and Bibliography:** The `custom-academic-template.cls` and `references.bib` files are treated as stable, immutable components (as per assignment rules), ensuring that stylistic and bibliographic consistency is maintained across all compilations.
- **Comprehensive Logging:** Detailed logs (in `ex04/logs/`) for content generation and QA checks provide an auditable trail of the document's creation, aiding in diagnosing any deviations during reproduction attempts.

By meticulously controlling these factors, the system aims to eliminate sources of variability, thereby guaranteeing that the generated PDF is a faithful and reproducible artifact of the agentic process.

F.2 Submission Readiness Checklist

A thorough submission readiness checklist ensures that all deliverables are complete, correctly formatted, and easily verifiable by the recipient.

- **Final PDF Generation:** Confirm that the `main.pdf` in the `ex04/` directory is the latest compiled version and meets all page count and quality requirements.
- **Clean Repository State:** Ensure the Git repository is in a clean state, with all relevant changes committed and no untracked files that are part of the submission. The `.gitignore` file should effectively exclude build artifacts.
- **Comprehensive Documentation:** Verify that the `README.md` at the project root and any in-line comments in code or skill definitions provide sufficient information for understanding, running, and evaluating the project.
- **Adherence to Assignment Constraints:** Double-check all strict rules, such as "modify only `ex04/main.tex`", "no new LaTeX packages", and "no Hebrew text", have been rigorously followed.
- **Included Logs and Reports:** Ensure that `ex04/logs/generation_log.md` and `ex04/logs/qa_report.md` are current and included, providing a summary of agent activities and quality checks.
- **Dependency Manifest:** Confirm `requirements.txt` is present and accurately lists all Python package dependencies required for the agent scripts.
- **Clarity of Purpose:** The entire submission clearly articulates the assignment's objectives, the agentic architecture, and the methodologies employed, as detailed in the main body and appendices of the report.

This dual focus on reproducibility and submission readiness ensures that the project not only delivers a high-quality academic report but also provides a transparent and verifiable account of its automated generation process.

G Appendix G: Submission Quality Review

A final quality review is an essential part of an agentic document generation workflow. The purpose of this review is to verify that the project is not only technically present in the repository, but also understandable, reproducible, and aligned with the assignment requirements. In this submission, the review focuses on the relationship between the planning documents, the LaTeX source file, the custom class template, the generated PDF, the skills, and the quality assurance materials.

The first review dimension is structural completeness. A reviewer should be able to open the repository and immediately understand the role of each file and folder. The Assignment 04 directory contains planning files, source files, skills, scripts, logs, and the generated PDF. This organization supports independent inspection and reduces the likelihood that important work will be overlooked during grading.

The second review dimension is reproducibility. The PDF is not only uploaded as a final artifact, but is also connected to source files that can regenerate it. The presence of a main source file, a custom class file, and a compilation script demonstrates that the report was produced through a repeatable process. This is especially important in an assignment that focuses on document generation rather than only on final formatting.

The third review dimension is evidence. The repository includes logs, quality assurance materials, and skill definitions. These files show that the workflow was treated as an agentic process rather than as a single static writing task. The evidence makes the submission easier to audit and helps connect the final PDF to the process that produced it.

H Appendix H: Evaluation Rubric Mapping

A strong submission can be evaluated by mapping each expected requirement to a visible artifact in the repository. The product requirement document explains the goal and scope of the assignment. The plan describes the work process. The task checklist translates the plan into concrete execution items. The README files explain the project to an external evaluator. Together, these documents create a clear path from requirements to implementation.

The LaTeX source file is the central artifact of the report generation process. It contains the academic structure, the section hierarchy, the discussion, the appendices, and the bibliography. The custom class file separates formatting decisions from the main content. This separation supports cleaner maintenance and demonstrates an understanding of reusable document design.

The skills directory represents the agentic layer of the assignment. Each skill file describes a focused responsibility such as content writing, LaTeX building, structure checking, bidirectional text awareness, or reference validation. This modular approach makes the workflow easier to inspect and easier to improve.

The quality assurance script provides an automated validation layer. It checks whether required files and folders exist and gives a clear result. Although the script is intentionally simple, it demonstrates the principle that generated work should be evaluated before submission.

I Appendix I: Risk Review and Mitigation

One risk in this type of project is that the final document may become disconnected from the repository structure. This risk is mitigated by describing the repository components inside the report and by maintaining README files that explain the same structure from a practical perspective. The report and the repository therefore support each other.

A second risk is compilation fragility. LaTeX workflows may depend on packages that are not installed on every machine. This project mitigates that risk by keeping the template simple and by avoiding unnecessary dependencies. When compilation issues appeared, the preferred solution was to simplify the document structure rather than introduce additional fragile requirements.

A third risk is over-reliance on automatic generation. Agentic tools can produce content quickly, but they can also introduce formatting mistakes or unsafe technical examples. The workflow therefore combines automated generation with human inspection, terminal validation, and repeated compilation attempts. This combination reflects a realistic use of AI-assisted development.

A fourth risk is unclear grading evidence. A repository can contain many files, but without documentation the evaluator may not understand their purpose. The README files, logs, skills, and QA materials are included to make the submission legible, auditable, and easier to evaluate.

J Appendix J: Final Readiness Statement

The project is ready for submission when several conditions are satisfied. First, the planning documents must exist and clearly explain the assignment goals. Second, the LaTeX source and custom class file must exist and compile successfully. Third, the generated PDF must be included in the repository. Fourth, the README files must explain the purpose, structure, setup process, limitations, and expected output.

This assignment demonstrates the use of agentic thinking in academic document production. The work is not limited to writing a report. It also includes defining roles, creating reusable instructions, validating outputs, and documenting the full process. The result is a repository that can be reviewed both as a technical project and as an academic deliverable.

The final submission reflects a complete workflow. Requirements were identified, planning artifacts were created, agent skills were defined, a LaTeX document was generated, quality assurance was performed, and the final output was committed to GitHub. This makes the project aligned with the educational goals of Assignment 04.

The current version is also designed to be extendable. Future versions could include additional figures, generated graphs, richer reference handling, and a more advanced orchestration script. However, the present version already demonstrates the core workflow: a reproducible academic PDF generated from structured source files and supported by documentation, skills, and QA evidence.